La Rioja | Educación y Empleo
larioja.org

IES COMERCIO

Departamento de Informática y Comunicaciones

IES Comercio
Paseo del Prior, 97
26004 Logroño
Teléfono: 941 23 39 80
E-mail: informatica@iescomercio.com
www.iescomercio.com/informatica

DESARROLLO WEB EN ENTORNO CLIENTE

UT2. Sintaxis y sentencias simples

Instrucciones generales

- Debes resolver los tres ejercicios utilizando JavaScript.
- Cada ejercicio se realizará en el archivo indicado.
- No cambies el nombre de los archivos, de las funciones ni la línea module.exports.
- Las funciones deben devolver el resultado mediante return.
- No utilices prompt, alert, document.write ni acceso al DOM.
- Puedes crear funciones auxiliares dentro del mismo archivo.
- Junto al examen se incluyen pruebas públicas. Puedes ejecutarlas tantas veces como necesites.
- La corrección utilizará también casos adicionales. Superar las pruebas públicas no garantiza por sí solo la puntuación máxima.

Cómo ejecutar las pruebas

Abre una terminal en la carpeta del examen y ejecuta:

```
node test.js
```

Las pruebas cargarán automáticamente Ejercicio1.js, Ejercicio2.js y Ejercicio3.js. No modifiques test.js.

Entregable

Entrega un único archivo ZIP llamado:

```
Examen_UT2_Nombre_Apellidos.zip
```

El ZIP debe contener exclusivamente:

```
Ejercicio1.js
Ejercicio2.js
Ejercicio3.js
```

IES COMERCIO
Departamento de Informática y Comunicaciones
IES Comercio
Paseo del Prior, 97
26004 Logroño
Teléfono: 941 23 39 80
E-mail: informatica@iescomercio.com
www.iescomercio.com/informatica

Ejercicio 1. Números casi escalonados (3 puntos)

Un número es escalonado cuando sus cifras, leídas de izquierda a derecha, aumentan siempre de forma estricta.

- 1358 es escalonado porque 1 < 3 < 5 < 8.
- 1337 no es escalonado porque contiene dos cifras iguales consecutivas.
- 421 no es escalonado porque sus cifras disminuyen.

Un número es casi escalonado cuando puede convertirse en un número escalonado eliminando exactamente una de sus cifras.

Crea la función:

```
esCasiEscalonado(numero)
```

La función recibe un número entero positivo de al menos dos cifras y debe devolver true si puede convertirse en escalonado eliminando exactamente una cifra, y false en caso contrario.

Reglas

- No se pueden cambiar de posición las cifras.
- Debe eliminarse exactamente una cifra.
- Un número de una sola cifra se considera escalonado.
- Si al eliminar la primera cifra quedan ceros a la izquierda, se ignoran al interpretar el número.

Ejemplos

```
esCasiEscalonado(1324); // true
// Eliminando el 3 se obtiene 124.

esCasiEscalonado(1243); // true
// Eliminando el 4 se obtiene 123.

esCasiEscalonado(1234); // true
// Aunque ya es escalonado, se elimina exactamente una cifra. Por ejemplo, 4.

esCasiEscalonado(4432); // false

esCasiEscalonado(1024); // true
// Eliminando el 1 se obtiene 024, que se interpreta como 24.
```

Archivo de entrega

```
Ejercicio1.js
```

IES Comercio
Paseo del Prior, 97
26004 Logroño
Teléfono: 941 23 39 80
E-mail: informatica@iescomercio.com
www.iescomercio.com/informatica

Ejercicio 2. Recorrido de un robot (3,5 puntos)

Un robot comienza en la posición x = 0, y = 0 y recibe un array de instrucciones. Cada instrucción es una cadena formada por una dirección (N, S, E u O) y un número entero positivo que indica cuántas posiciones debe avanzar.

- N: aumenta y.
- S: disminuye y.
- E: aumenta x.
- O: disminuye x.

El robot avanza una posición cada vez. Por ejemplo, E3 representa tres movimientos consecutivos hacia el este.

Crea la función:

```
recorrerRobot(instrucciones)
```

La función debe devolver:

```
{
  x: numero,
  y: numero,
  distanciaMaxima: numero,
  primeraRepetida: array_o_null
}
```

- x e y son las coordenadas finales.
- distanciaMaxima es la mayor distancia Manhattan alcanzada respecto al origen durante el recorrido. Se calcula como |x| + |y|.
- primeraRepetida contiene la primera posición visitada por segunda vez. Si no se repite ninguna posición, vale null.
- La posición inicial [0, 0] cuenta como visitada.
- Tras encontrar la primera posición repetida, el robot continúa ejecutando todas las instrucciones.
- La cantidad puede tener más de una cifra, por ejemplo N12 u O25.
- Puedes suponer que todas las instrucciones tienen un formato correcto.

Ejemplos

```
recorrerRobot(["N2", "E3", "S1"]);
// { x: 3, y: 1, distanciaMaxima: 5, primeraRepetida: null }

recorrerRobot(["N2", "E1", "S2", "O1"]);
// { x: 0, y: 0, distanciaMaxima: 3, primeraRepetida: [0, 0] }

recorrerRobot(["E3", "O2", "N1"]);
// { x: 1, y: 1, distanciaMaxima: 3, primeraRepetida: [2, 0] }
```

Archivo de entrega

```
Ejercicio2.js
```

IES Comercio
Departamento de Informática y Comunicaciones
IES Comercio
Paseo del Prior, 97
26004 Logroño
Teléfono: 941 23 39 80
E-mail: informatica@iescomercio.com
www.iescomercio.com/informatica

Ejercicio 3. Eliminación repetida de bloques (3,5 puntos)

Crea la función:

```
eliminarBloques(texto, cantidad)
```

La función recibe una cadena texto y un número entero cantidad. Cada vez que aparezcan cantidad caracteres iguales consecutivos, esos caracteres deben eliminarse.

Después de una eliminación pueden unirse caracteres que antes estaban separados y formar un nuevo bloque que también deba eliminarse. El proceso continúa hasta que no sea posible realizar más eliminaciones.

La función debe devolver la cadena resultante.

Reglas

- La cadena puede estar vacía.
- La comparación distingue mayúsculas y minúsculas.
- cantidad siempre será un entero mayor o igual que 2.
- Si un bloque contiene más caracteres que cantidad, se eliminan grupos de tamaño cantidad. Por ejemplo, cuatro letras a con cantidad 3 dejan una a.
- No es suficiente realizar una única pasada.

Ejemplos

```
eliminarBloques("aaabbb", 3); // ""

eliminarBloques("deeedbbcccbdaa", 3); // "aa"

eliminarBloques("abcd", 2); // "abcd"

eliminarBloques("aabbbaa", 3); // "a"
// Se eliminan bbb, quedan aaaa y se eliminan tres a.

eliminarBloques("ppqqqpp", 3); // "p"
```

Archivo de entrega

```
Ejercicio3.js
```